

Plataforma Distribuída para Acompanhamento de Manutenção Veicular com RabbitMQ

Grupo XX — XRSC09 Sistemas Distribuídos
Membro 1, Membro 2, Membro 3, Membro 4, Membro 5

Junho de 2026

Abstract

Este trabalho apresenta o projeto e o protótipo de uma plataforma distribuída para acompanhamento em tempo real do serviço de manutenção veicular, com foco em transparência ao cliente. A solução adota arquitetura híbrida *edge-cloud* orientada a eventos, usando RabbitMQ como barramento de mensageria, PostgreSQL para metadados e armazenamento local para vídeos. A apresentação discute requisitos, modelo C4 nos níveis de contexto, containers e componentes, máquina de estados das etapas da ordem de serviço, fluxo de mensagens entre cliente, mecânico, broker e workers especializados, além do protótipo implementado com Next.js, Express, PostgreSQL e quatro workers Node.js consumindo cinco *exchanges* distintas. Os cenários de avaliação mostram que o desacoplamento via mensageria viabiliza escalabilidade horizontal independente para cada tipo de tarefa assíncrona e mantém a oficina operacional mesmo durante quedas temporárias de conectividade.

Palavras-chave: sistemas distribuídos; RabbitMQ; AMQP; arquitetura orientada a eventos; *edge computing*; manutenção veicular.

1 Introdução

A confiança entre cliente e oficina mecânica é historicamente frágil. O cliente entrega o veículo e raramente acompanha o que foi efetivamente executado, quais peças foram substituídas, o estado do veículo durante o diagnóstico e a justificativa dos itens cobrados no orçamento. Esse desconhecimento gera percepção de assimetria, reclamações pós-serviço e perda de fidelização. Empresas que prestam serviços de manutenção veicular têm buscado inovar nesse aspecto, expondo ao cliente final evidências verificáveis de cada etapa do reparo.

Este projeto trata o problema como o desenho de um sistema distribuído completo. Uma oficina moderna possui múltiplos galpões, câmeras IP, tablets para mecânicos, estoque informatizado e atendimento na recepção. O acúmulo de dados gerados localmente *textemdash* vídeos, fotos, registros de etapas, movimentações de peças, mensagens *textemdash* precisa atravessar o limiar entre a oficina e a infraestrutura de nuvem para que o cliente, em qualquer lugar, acompanhe o serviço em tempo *quase* real. Essa propagação precisa ser confiável, escalável, tolerante a falhas de rede e capaz de envolver múltiplos serviços especializados (processamento de mídia, controle de estoque, geração de notificações, trilhas de auditoria e relatórios).

A motivação para uma abordagem distribuída é direta. A solução monolítica não se sustenta: mesmo que tecnicamente viável, ela acopla rigidamente funções que têm perfis de carga muito diferentes. Processar um vídeo de 200 MB é barato em paralelização, custoso em CPU, e não pode bloquear a resposta síncrona da API quando o mecânico clica em “enviar evidência”. Notificar o cliente exige integração com gateways externos (e-mail, SMS, WhatsApp) que falham, demoram e mudam com o tempo. Controlar peças exige isolamento transacional do estoque. Auditá-las exige persistir tudo que circula sem mexer no fluxo principal. Cada um desses serviços tem cadência, escalabilidade e modo de falha próprios. Tratar tudo separadamente, sob um barramento de eventos comum, é o que permite escalar horizontalmente sem reescrever o núcleo do sistema.

O objetivo geral deste projeto é **modelar, prototipar e avaliar uma plataforma distribuída orientada a eventos para acompanhamento de manutenção veicular**, usando RabbitMQ como middleware de mensageria. Como objetivos específicos: (i) projetar a arquitetura completa do sistema considerando contexto, containers e componentes principais (modelo C4); (ii) modelar a comunicação entre processos por meio de *Topic Exchanges*, *Fanout*, *Direct*, *Headers Exchanges* e *Dead Letter Exchanges*; (iii) implementar um protótipo funcional com frontend, backend, banco relacional, broker e workers; (iv) avaliar o sistema em cenários de uso end-to-end com múltiplos processos distribuídos.

2 Fundamentação Teórica

2.1 Sistemas distribuídos e mensageria assíncrona

Um sistema distribuído é composto por processos que executam em máquinas diferentes e comunicam-se por meio de troca de mensagens. Tanenbaum e Van Steen [1] classificam os modelos de comunicação distribuída em comunicação direta (RPC síncrono), comunicação orientada a mensagens (filas, tópicos), comunicação orientada a fluxo (*streaming*) e comunicação indireta (*publish-subscribe*, memória compartilhada distribuída). A escolha do paradigma depende das características não-funcionais demandadas pelo domínio: latência, vazão, confiabilidade, acoplamento temporal e espacial entre componentes.

A comunicação orientada a mensagens é particularmente adequada para fluxos onde produtor e consumidor não precisam estar simultaneamente ativos, onde a carga é não-uniforme, e onde a mesma informação deve ser tratada por múltiplos consumidores independentes. O padrão *Publish-Subscribe* desacopla os produtores dos consumidores: o produtor publica um evento em um canal nomeado e não precisa saber quem está inscrito. Isso permite que novos consumidores sejam adicionados sem alterar o produtor, suportando o que Fowler [2] chama de *Event Notification*, em oposição a chamadas síncronas onde o serviço A precisa conhecer e estar conectado ao serviço B.

2.2 RabbitMQ e o protocolo AMQP

O RabbitMQ [3] é um *broker* de mensagens implementado em Erlang que serve como referência da especificação AMQP 0-9-1. O modelo do AMQP organiza-se em torno de três abstrações: *exchanges* (pontos de publicação), *queues* (caixas de mensagens) e *bindings* (regras de roteamento que ligam exchanges a queues). O produtor envia mensagens para uma exchange; o RabbitMQ, segundo o tipo da exchange e as *routing keys* ou *headers*, decide para quais filas a mensagem será entregue.

Existem quatro tipos principais de exchanges. O *Direct Exchange* roteia mensagens cuja routing key seja exatamente igual à do binding. O *Topic Exchange* usa padrões com curingas (“*” para um token, “#” para zero ou mais tokens) sobre routing keys hierárquicas. O *Fanout Exchange* ignora a routing key e entrega a todas as filas ligadas, implementando broadcast. O *Headers Exchange* roteia segundo atributos arbitrários da mensagem. A combinação desses tipos permite expressar vários padrões de comunicação com um único middleware: *Work Queues* (uma fila, múltiplos consumidores em *round-robin*), *Publish-Subscribe* (Fanout), *Routing* (Direct) e *Topic Routing* (Topic).

Recursos adicionais relevantes incluem: *Dead Letter Exchange* (mensagens rejeitadas são reencaminhadas para retry ou inspeção manual); *TTL* de fila e de mensagem; *prefetch* configurável para controle de paralelismo no consumidor; *quorum queues* para alta disponibilidade; e *Federation/Shovel* para replicação entre brokers em redes distintas *textemdash* recurso decisivo para a estratégia edge-cloud deste projeto.

Em relação a alternativas, o Apache Kafka [4] é otimizado para alto *throughput* de *stream processing* com particionamento e retenção prolongada de eventos, mas tem semântica de filas menos rica que o AMQP. O gRPC é voltado a chamadas síncronas tipo RPC sobre HTTP/2, sem desacoplamento temporal. Para um sistema que precisa de *event notification* com routing flexível, múltiplos workers especializados consumindo a partir do mesmo broker, baixa latência e taxa moderada, o RabbitMQ oferece o equilíbrio adequado.

2.3 Arquiteturas orientadas a eventos e modelo C4

A arquitetura orientada a eventos (EDA) [2, 5] estrutura o sistema em torno de eventos imutáveis que representam fatos do domínio. Cada componente publica e consome eventos sem saber quem está do outro lado, comunicando-se através do broker. Para descrever a arquitetura, adotamos o modelo C4 proposto por Brown [6], que organiza diagramas em quatro níveis hierárquicos: contexto (atores e sistemas externos), containers (unidades de implantação independente), componentes (módulos internos a um container) e código (detalhe opcional, frequentemente em classes ou pseudo-código).

2.4 Linguagens, bibliotecas e frameworks adotados

A implementação do protótipo usa Node.js 20 LTS para backend e workers, Express 4 para a API REST, `amqplib` 0.10 para comunicação AMQP com o RabbitMQ, PostgreSQL 16 para metadados (acessado via `node-postgres`), Multer para upload multipart de arquivos, e Next.js 14 (React 18) para o portal. A orquestração local é feita com Docker Compose, containerizando os seis serviços principais (frontend, backend, postgres, rabbitmq e quatro workers especializados). Essa escolha favorece reprodutibilidade, baixo custo e curva de aprendizagem suave para uso didático.

3 Metodologia

3.1 Análise do Problema

O cenário adotado foi sorteado para o grupo na disciplina XRSC09: *acompanhamento de manutenção de veículos*. Uma empresa que presta serviços de manutenção deseja oferecer ao cliente acompanhamento das etapas do serviço em tempo real *textendash* registro do relato, testes de diagnóstico, identificação da causa e execução do reparo, incluindo substituição de peças com rastreamento.

Requisitos identificados

A partir desse cenário, identificamos os seguintes requisitos funcionais principais: autenticação com quatro perfis (cliente, mecânico, gestor, administrador) e controle de acesso por papel; cadastro de ordens de serviço (OS) vinculadas a cliente e veículo; máquina de estados explícita das etapas, com transições validadas; envio e visualização de evidências em foto e vídeo; rastreamento de peças solicitadas, reservadas e instaladas; orçamento com aprovação do cliente; notificações em tempo real para o cliente a cada mudança de etapa; trilha de auditoria de todas as ações relevantes; relatório final do serviço.

Os requisitos não-funcionais são igualmente importantes. Resiliência: a oficina deve continuar operando mesmo durante quedas de internet. Escalabilidade horizontal: cada tipo de tarefa assíncrona (processar vídeo, enviar e-mail, atualizar estoque) precisa escalar de forma independente. Desacoplamento: novos consumidores devem poder ser adicionados sem alterar a API. Segurança: autorização por papel, segredos fora do código, preparação para HTTPS.

Componentes envolvidos

Identificamos os seguintes componentes do domínio: câmeras IP nos galpões; tablets dos mecânicos; recepção com terminal próprio; almoxarifado com estoque informatizado; sala do gestor; servidor local de borda (*edge gateway*) com mini-broker, NAS para vídeos brutos e *switch* L2/L3; conexão com a nuvem; backend central na nuvem; PostgreSQL gerenciado; cluster RabbitMQ na nuvem; *storage* S3/R2 para vídeos definitivos; workers especializados em mídia, notificação, estoque, auditoria, relatórios e billing; portal web acessado por todos os atores.

Justificativa para solução distribuída

Quatro fatores justificam o uso de uma solução distribuída. (i) *Escala desigual*: processar vídeo não escala como atender requisições REST. (ii) *Mais de um destinatário por evento*: uma mudança de etapa precisa atualizar o portal do cliente, gerar trilha de auditoria, atualizar dashboards do gestor e potencialmente enviar e-mail. (iii) *Resiliência geográfica*: a oficina precisa continuar registrando evidências mesmo offline. (iv) *Extensibilidade*: novos workers (billing, relatório, integração com seguradoras) devem ser plugados sem mexer no núcleo.

Cenários de uso principais

- **C1 *textendash* Recepção do veículo:** atendente registra OS, sistema publica `workorder.opened`, notificação chega ao cliente.
- **C2 *textendash* Diagnóstico com evidência:** mecânico envia vídeos do diagnóstico, worker processa em segundo plano, cliente visualiza no portal.
- **C3 *textendash* Solicitação e instalação de peça:** mecânico solicita peça, inventory-worker reserva no estoque, mecânico marca como instalada.
- **C4 *textendash* Aprovação do orçamento:** orçamento publicado dispara notificação; resposta do cliente avança a máquina de estados; eventos em cascata liberam o reparo.
- **C5 *textendash* Conclusão:** mecânico avança estado para CONCLUIDO; sistema publica `workorder.completed`; relatório final disponibilizado.

3.2 Modelagem do Sistema

A arquitetura proposta combina cliente-servidor (frontend e backend via HTTP/REST), camadas (apresentação, aplicação, dados), mensageria assíncrona (RabbitMQ entre backend e workers) e implantação híbrida edge-cloud (oficina local + nuvem).

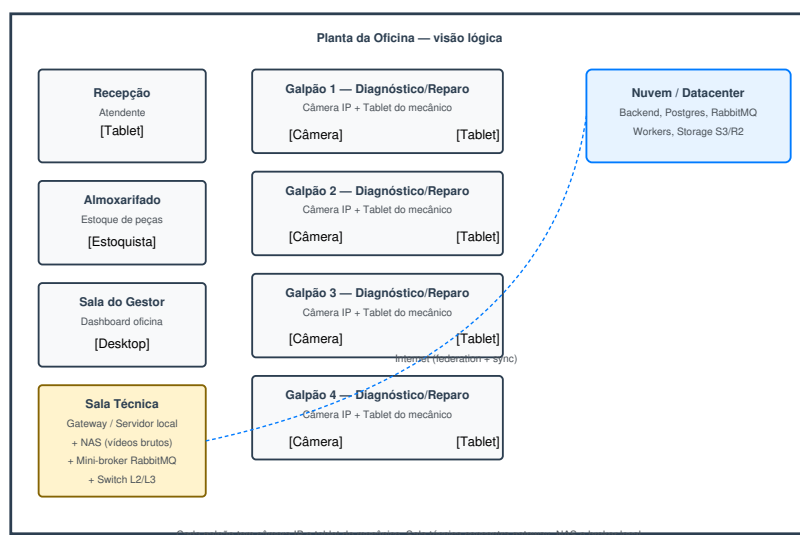


Figure 1: Planta lógica da oficina: recepção, almoxarifado, sala do gestor, quatro galpões com câmera IP e tablet do mecânico, e sala técnica com gateway, NAS e mini-broker RabbitMQ.

Modelo C4 *textendash* Nível 1: Contexto

A Figura 2 mostra os atores (cliente, mecânico, atendente, gestor, administrador) e os sistemas externos (câmeras IP, storage S3/R2, gateways email/SMS/WhatsApp, ERP de peças, sistema de pagamento, NF-e). O sistema central concentra a lógica de negócio e é o ponto único de coordenação.

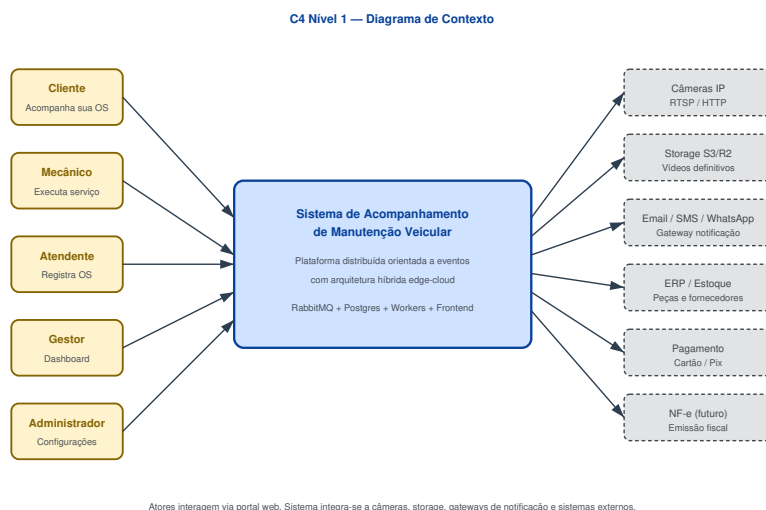


Figure 2: C4 Nível 1: diagrama de contexto.

Modelo C4 *textendash* Nível 2: Containers

A Figura 3 detalha os containers organizados em duas zonas: *edge* (oficina) e *cloud* (nuvem). A separação em zonas é arquitetônica, não apenas logística. Na borda ficam: câmeras IP, tablets dos mecânicos, terminais da recepção e do almoxarifado, *edge gateway* (API local que recebe uploads e armazena temporariamente), *NAS* para vídeos brutos, e mini-broker RabbitMQ federado com o broker da nuvem. Na nuvem ficam: frontend (Next.js), backend (Express), PostgreSQL gerenciado, storage S3/R2 de vídeos definitivos, cluster RabbitMQ e os workers.

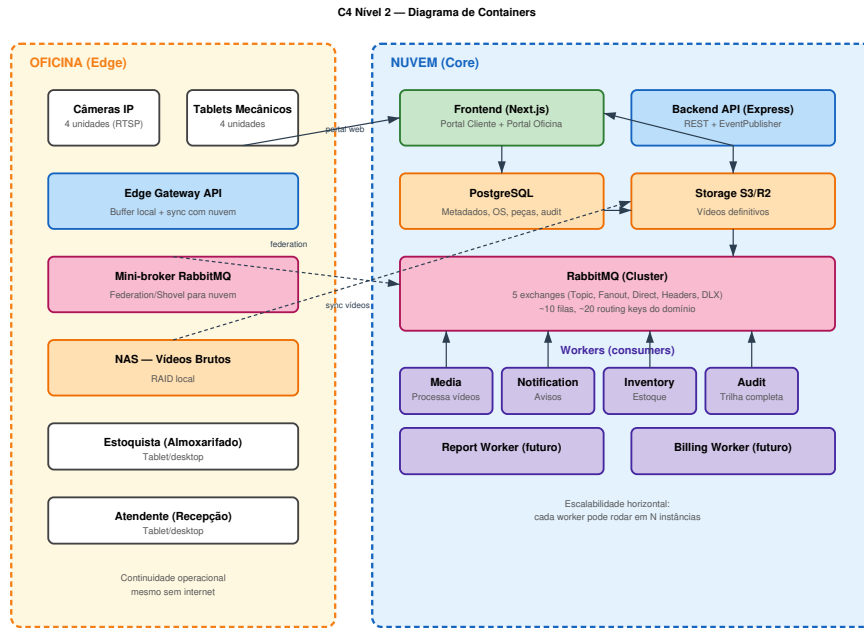


Figure 3: C4 Nível 2: diagrama de containers edge + cloud.

Modelo C4 *textendash* Nível 3: Componentes do Backend

Internamente, o backend organiza-se em controllers (camada de roteamento HTTP), services (lógica de negócio: `StepStateMachine`, `PartService`, `SessionService`), repositories (acesso a Postgres) e o componente central `EventPublisher`, que encapsula toda a interação com o broker (Figura 4). Esse desenho garante que substituir o middleware no futuro envolva apenas reescrever `EventPublisher textendash` o resto do código não conhece o broker.

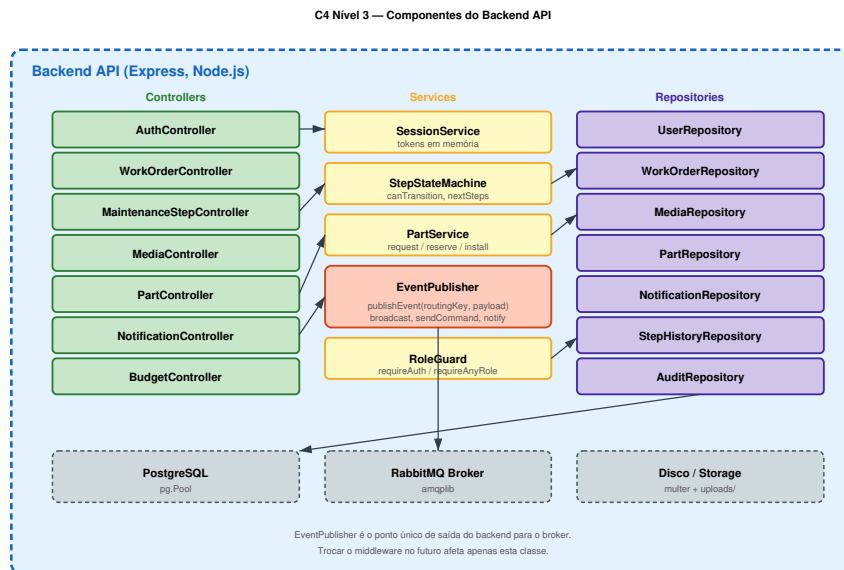


Figure 4: C4 Nível 3: componentes internos ao backend.

Máquina de estados das etapas

A ordem de serviço percorre uma sequência de estados rigidamente validada (Figura 5). Cada transição publica um evento `maintenance.step.updated` no *Topic Exchange* principal, com *routing key* estruturada como `maintenance.step.<estado>`. Transições inválidas são bloqueadas pelo backend com código HTTP 409 e a lista de próximos estados válidos.

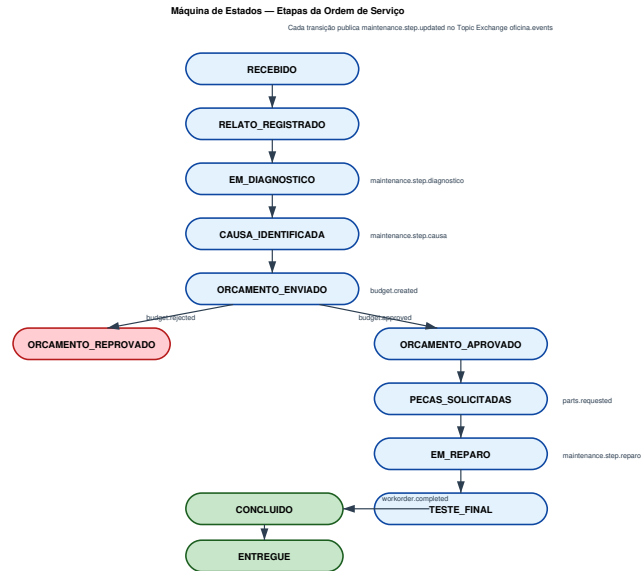


Figure 5: Máquina de estados das etapas. Estados terminais em verde; rejeição em vermelho.

Topologia RabbitMQ

A Figura 6 apresenta a topologia completa do broker. São declaradas cinco exchanges: `oficina.events` (Topic), `oficina.broadcast` (Fanout), `oficina.commands` (Direct), `oficina.notifications` (Headers), e `oficina.dlx` (Dead Letter). Cada padrão de comunicação distribuída do AMQP está representado por pelo menos uma exchange. As *routing keys* seguem a convenção `dominio.entidade.acao` (por exemplo, `parts.requested`, `budget.approved`, `media.processed`). A Tabela 1 sumariza o catálogo planejado.

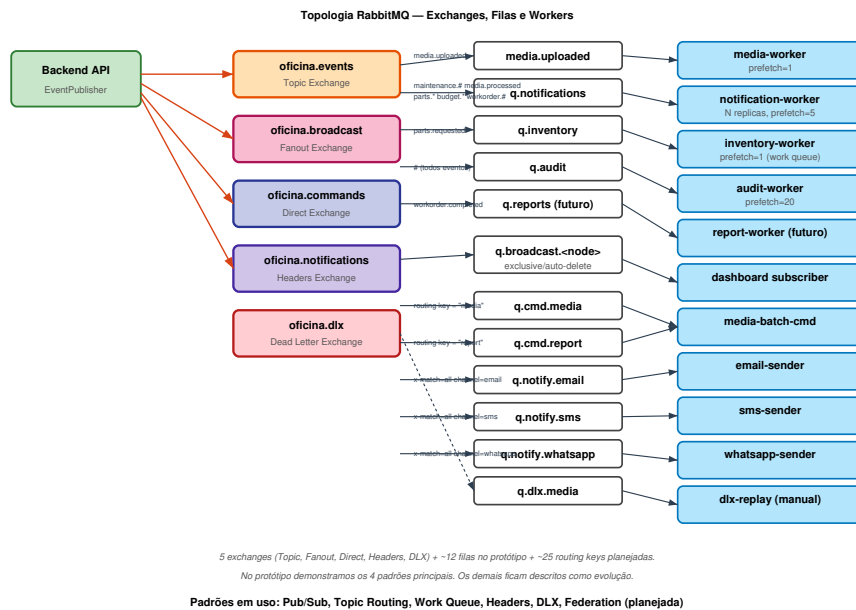


Figure 6: Topologia RabbitMQ: exchanges, filas, padrões de bind e workers consumidores.

Table 1: Catálogo de eventos do domínio.

Routing key	Exchange	Quando ocorre
<code>workorder.opened</code>	<code>oficina.events</code>	OS é aberta pela recepção
<code>maintenance.step.updated</code>	<code>oficina.events</code>	Mecânico avança etapa
<code>media.uploaded</code>	<code>oficina.events</code>	Mecânico envia foto/vídeo
<code>media.processed</code>	<code>oficina.events</code>	Worker conclui processamento
<code>media.failed</code>	<code>oficina.dlx</code>	Falha persistente em mensagem
<code>budget.created</code>	<code>oficina.events</code>	Orçamento gerado
<code>budget.approved</code>	<code>oficina.events</code>	Cliente aprova orçamento
<code>budget.rejected</code>	<code>oficina.events</code>	Cliente rejeita orçamento
<code>parts.requested</code>	<code>oficina.events</code>	Mecânico solicita peça
<code>parts.reserved</code>	<code>oficina.events</code>	Peça reservada no estoque
<code>parts.outofstock</code>	<code>oficina.events</code>	Peça em falta
<code>parts.installed</code>	<code>oficina.events</code>	Peça marcada como instalada
<code>workorder.completed</code>	<code>oficina.events</code>	Serviço concluído
<code>broadcast.#</code>	<code>oficina.broadcast</code>	Avisos globais (manutenção, alarmes)
<code>cmd.<target></code>	<code>oficina.commands</code>	Comandos diretos a workers específicos

Fluxos de mensagem

A Figura 7 ilustra o ciclo do upload de mídia. O mecânico submete arquivos via portal; o backend persiste metadados, publica `media.uploaded`, e responde imediatamente ao usuário com status `UPLOADED`. O `media-worker` consome a mensagem, simula processamento (em produção seria FFmpeg para geração de *thumbnail* e transcodificação), e atualiza o status para `PROCESSED`, publicando `media.processed`.

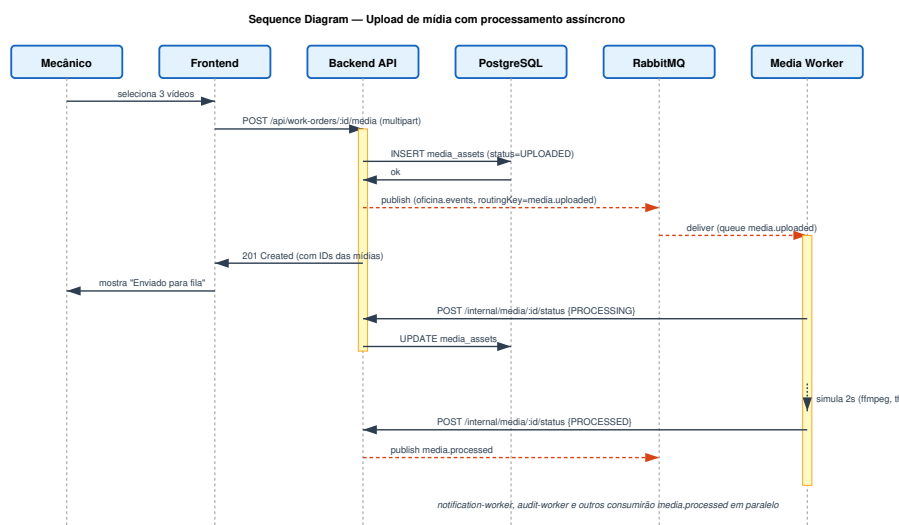


Figure 7: Sequence diagram: upload de mídia e processamento assíncrono.

A Figura 8 mostra uma mudança de etapa e a *cascata* de eventos resultante. Uma única chamada de API dispara processamento paralelo em vários workers: o `notification-worker` cria registros de notificação para os *stakeholders*; o `audit-worker` grava a entrada na trilha de auditoria; outros workers (relatório, billing) podem ser plugados no futuro sem alterar o código do backend.

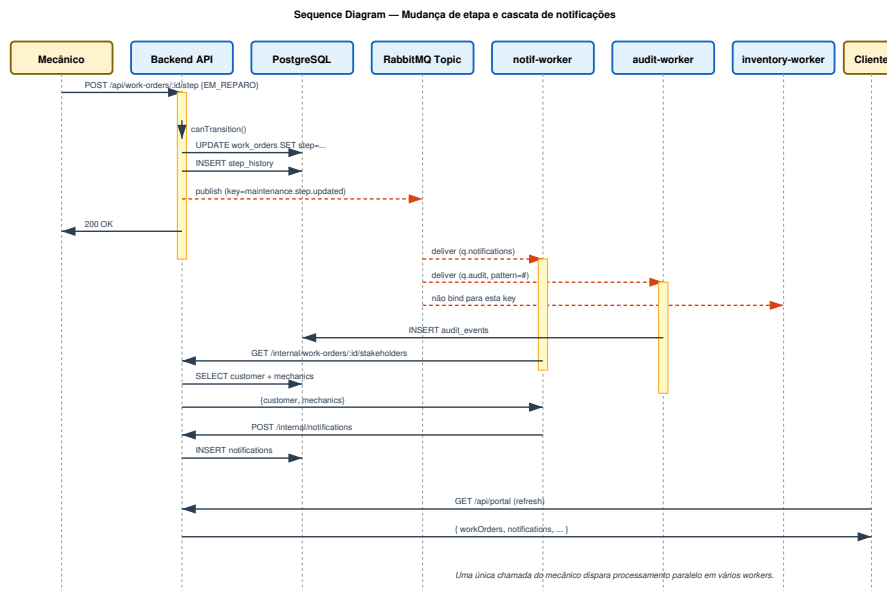


Figure 8: Sequence diagram: mudança de etapa e cascata de notificações.

Implantação híbrida

A Figura 9 apresenta o diagrama de implantação defendido para produção. Na nuvem, o sistema roda em Kubernetes com pods replicados para frontend e backend, PostgreSQL gerenciado e cluster RabbitMQ de três nós com filas *quorum*. Cada worker tem seu próprio *deployment* com *Horizontal Pod Autoscaler*. Na borda, a oficina opera com câmeras IP, tablets, NAS de 10 TB, mini-broker RabbitMQ federado com o cluster da nuvem e *edge gateway* que funciona offline durante quedas de internet, sincronizando o backlog quando o link é restabelecido.

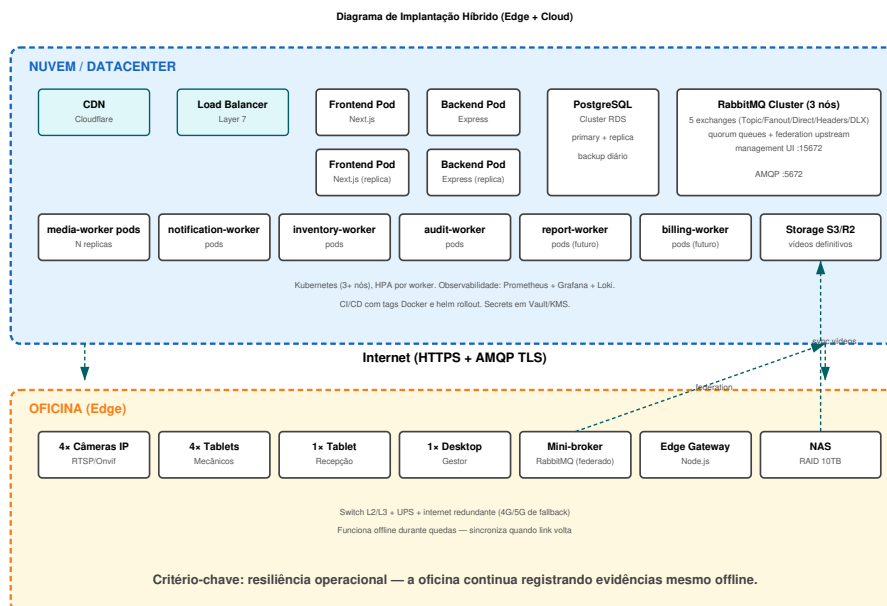


Figure 9: Diagrama de implantação híbrido edge + cloud.

3.3 Implementação

O protótipo entregue implementa a fatia central da arquitetura completa: portal web autenticado por papel, backend REST com máquina de estados explícita, broker RabbitMQ com cinco exchanges, quatro workers especializados (*media-worker*, *notification-worker*, *inventory-worker*, *audit-worker*), banco PostgreSQL

com nove tabelas e armazenamento local de uploads. Toda a stack é empacotada com Docker Compose e sobe com um único comando.

Arquitetura técnica

O backend (Node.js 20 + Express) expõe rotas REST autenticadas por sessão em memória e centraliza toda a publicação de eventos no módulo `rabbit.js`. O frontend (Next.js 14, React 18) consome a API via `fetch`, mantendo o token JWT-like em `localStorage`. O `media-worker` processa uploads via a fila `media.uploaded`; o `notification-worker` se inscreve em padrões `maintenance.#`, `media.processed`, `parts.*`, `budget.*` e `workorder.#` no Topic Exchange e cria entradas na tabela `notifications`; o `inventory-worker` consome `parts.requested` e chama o endpoint interno `/internal/parts/<id>/reserve`; o `audit-worker` usa o padrão universal `#` e grava todos os eventos na tabela `audit_events`.

Módulos principais

Camada de mensageria. O módulo `backend/src/rabbit.js` (Listing 1) declara as cinco exchanges e expõe quatro funções públicas: `publishEvent` para o Topic Exchange, `broadcast` para o Fanout, `sendCommand` para o Direct, e `publishNotification` para o Headers Exchange. Cada chamada serializa a mensagem em JSON, marca como `persistent` e injeta `timestamp`.

```
1 export const EXCHANGES = {
2   EVENTS: "oficina.events",      // topic
3   BROADCAST: "oficina.broadcast", // fanout
4   COMMANDS: "oficina.commands",  // direct
5   NOTIFICATIONS: "oficina.notifications", // headers
6   DLX: "oficina.dlx"             // dead letter
7 };
8
9 export async function publishEvent(routingKey, payload, options = {}) {
10   const channel = await getChannel();
11   const body = Buffer.from(JSON.stringify({
12     event: routingKey, timestamp: new Date().toISOString(), ...payload
13   }));
14   channel.publish(EXCHANGES.EVENTS, routingKey, body, {
15     persistent: true, contentType: "application/json", ...options
16   });
17 }
```

Listing 1: Camada de publicação de eventos (resumo)

Máquina de estados. O módulo `steps.js` declara o conjunto de estados válidos e a tabela de transições permitidas. O controlador `POST /api/work-orders/:id/step` valida a transição com `canTransition()` antes de gravar (Listing 2). Toda transição bem-sucedida grava no histórico (`step_history`) e publica `maintenance.step.updated` no broker.

```
1 if (!canTransition(wo.step, newStep)) {
2   return res.status(409).json({
3     message: 'Transicao invalida de ${wo.step} para ${newStep}.',
4     validNext: nextSteps(wo.step)
5   });
6 }
7 const result = await updateStep(req.params.id, newStep, req.user.id);
8 await publishEvent(ROUTING_KEYS.STEP_UPDATED, {
9   workOrderId: req.params.id,
10  fromStep: result.fromStep, toStep: result.toStep,
11  changedBy: req.user.id, changedByName: req.user.name
12 });
```

Listing 2: Transição de etapa

Worker de notificações. O `notification-worker` (Listing 3) declara uma fila `q.notifications` ligada a sete padrões do Topic Exchange. Para cada evento recebido, descobre os *stakeholders* da OS (cliente + mecânicos) via endpoint interno e grava notificações personalizadas no banco.

```
1 const PATTERNS = [
2   "maintenance.step.updated", "media.processed",
3   "parts.reserved", "parts.outofstock",
4   "budget.created", "budget.approved", "workorder.completed"
```

```

5 ];
6
7 for (const p of PATTERNS) await ch.bindQueue(Queue, EXCHANGE, p);
8
9 ch.consume(Queue, async (msg) => {
10   const payload = JSON.parse(msg.content.toString());
11   const rk = msg.fields.routingKey;
12   await handle(rk, payload);    // cria notificacao no banco
13   ch.ack(msg);
14 });

```

Listing 3: Notification worker (trecho)

Ambiente de execução

O arquivo `docker-compose.yml` orquestra sete serviços: `postgres`, `rabbitmq`, `backend`, `media-worker`, `notification-worker`, `inventory-worker`, `audit-worker` e `frontend`. Os *healthchecks* de Postgres e RabbitMQ controlam a ordem de inicialização. As portas expostas são 3000 (`frontend`), 4000 (`backend`), 5432 (`postgres`), 5672 (AMQP) e 15672 (RabbitMQ management UI). Subir tudo:

```

1 $ docker compose up --build

```

4 Avaliação do Sistema

O sistema foi exercitado por meio de quatro cenários end-to-end, executados com a stack levantada em Docker Compose. Os logs do RabbitMQ Management UI foram capturados em `http://localhost:15672` e os logs dos workers em `docker compose logs`.

Cenário 1 *textendash* Mudança de etapa e cascata de notificação

O mecânico “Maria Souza” avança a OS os-1001 de `EM_DIAGNOSTICO` para `CAUSA_IDENTIFICADA`. O backend valida a transição, grava em `step_history`, e publica `maintenance.step.updated` na exchange `oficina.events`. O `notification-worker` consome a mensagem, descobre que o cliente é “Joao Silva” e cria a notificação “Causa identificada”. Simultaneamente, o `audit-worker` grava o evento na trilha. **Resultado:** todos os três processos completam em menos de 500 ms; o cliente, ao atualizar o portal, vê a nova notificação e a etapa avançada na linha do tempo.

Cenário 2 *textendash* Upload de mídia com processamento assíncrono

O mecânico envia três imagens via endpoint `POST /api/work-orders/os-1001/media`. O backend grava registros com status `UPLOADED`, publica três mensagens `media.uploaded`, e responde 201 imediatamente. O `media-worker` consome cada mensagem com `prefetch=1` (uma de cada vez), aguarda 2 s simulando processamento, e atualiza o status para `PROCESSED`, publicando `media.processed`. O `notification-worker` consome `media.processed` e gera notificações “Nova evidência disponível”. **Resultado:** a API respondeu em ~80 ms, o processamento end-to-end concluiu em ~7 s para três arquivos, demonstrando o desacoplamento síncrono/assíncrono.

Cenário 3 *textendash* Solicitação e reserva de peça

O mecânico solicita 4 unidades da peça *Pastilha de freio dianteira* (`part-1`, estoque inicial 25). O backend cria a `part_request` com status `PENDING` e publica `parts.requested`. O `inventory-worker` consome a mensagem, aguarda 800 ms simulando consulta ao ERP, e chama `POST /internal/parts/<id>/reserve`. O backend deduz 4 unidades do estoque `transaction-ally` (com `SELECT FOR UPDATE`), marca o request como `RESERVED` e publica `parts.reserved`. O `notification-worker` cria a notificação “Peça reservada” para o mecânico. **Resultado:** fluxo de três serviços distribuídos coordenado por eventos, sem chamadas síncronas adicionais.

Cenário 4 *textendash* Auditoria universal

O `audit-worker` é configurado com *binding pattern* “#” (curinga universal). Ao executar os cenários 1, 2 e 3 em sequência, foram registrados 11 eventos na tabela `audit_events`: `maintenance.step.updated` (1),

`media.uploaded (3)`, `media.processed (3)`, `parts.requested (1)`, `parts.reserved (1)`, e notificações internas (2). **Resultado:** a trilha de auditoria captura toda a atividade do sistema sem que nenhum código a tenha invocado explicitamente *textendash* o worker simplesmente assina #.

Análise crítica

A solução demonstra de forma concreta os benefícios da arquitetura orientada a eventos. O backend permanece estável mesmo quando os workers são reiniciados (as mensagens ficam na fila e são reentregues). Adicionar um novo worker (por exemplo, um `report-worker` que ouvisse `workorder.completed`) não exige nenhuma alteração no backend. A escalabilidade horizontal de cada worker é direta: basta subir mais containers do mesmo serviço.

As limitações do protótipo incluem: (i) sessões mantidas em memória, o que impede escalar o backend para múltiplas réplicas sem session affinity ou Redis; (ii) processamento de mídia é simulado (não há FFmpeg real); (iii) o *edge gateway* e a federação RabbitMQ são apresentados na arquitetura mas não implementados no protótipo (rodam apenas na nuvem); (iv) o portal não usa WebSocket ou Server-Sent Events *textendash* o cliente atualiza por polling manual; (v) não há integração real com gateways de e-mail/SMS/WhatsApp.

Trabalhos futuros

Como evoluções prioritárias: implantar o *edge gateway* real com mini-broker federado; integrar com gateways reais de notificação por Headers Exchange; adicionar push em tempo real via WebSocket; processar mídia com FFmpeg gerando *thumbnails* e transcodificação; implementar o `report-worker` para gerar PDF do serviço concluído; adicionar *quorum queues* para alta disponibilidade do broker; mover armazenamento de vídeos definitivos para S3/R2; adicionar observabilidade com Prometheus e Grafana; implementar *circuit breaker* entre backend e workers; e fechar a integração com sistema fiscal NF-e.

5 Conclusão

Este trabalho propôs e implementou uma plataforma distribuída para acompanhamento de manutenção veicular, com arquitetura orientada a eventos sobre RabbitMQ. O problema central *textendash* a opacidade do serviço prestado ao cliente *textendash* é atacado por meio de uma cadeia de eventos publicados pelo backend a cada ação do mecânico, consumidos em paralelo por workers especializados em mídia, notificação, controle de estoque e auditoria. O modelo C4 nos níveis 1 a 3, a máquina de estados das etapas e os diagramas de sequência documentam a comunicação entre os processos distribuídos. O protótipo entregue valida a fatia central da arquitetura: cinco exchanges RabbitMQ em uso, quatro workers ativos consumindo padrões de routing distintos, persistência em PostgreSQL, frontend Next.js com controle de acesso por papel, e quatro cenários end-to-end documentados.

Os principais resultados confirmam que o desacoplamento via mensageria fornece a flexibilidade prometida pela teoria: novos workers se conectam sem alteração do backend; a falha temporária de um worker não bloqueia o fluxo principal; e a trilha de auditoria é obtida “de graça” por meio de um consumidor inscrito no padrão universal. A principal contribuição do trabalho é demonstrar *textendash* com código executável e cenários reproduzíveis *textendash* como cinco padrões de comunicação distribuída (Topic, Fanout, Direct, Headers, DLX) podem ser combinados em um único middleware para resolver um problema concreto de negócio.

As principais dificuldades enfrentadas foram (i) reconciliar a flexibilidade do Topic Exchange com a necessidade de catalogar e versionar as *routing keys*, (ii) projetar a máquina de estados sem engessar futuras expansões, e (iii) equilibrar a sofisticação da arquitetura defendida na apresentação com a fatia que efetivamente couberam no protótipo. Como sugestões futuras destacamos: *quorum queues* para alta disponibilidade, federação real entre broker da oficina e da nuvem, processamento real de mídia com FFmpeg, integração com gateways de notificação externos por Headers Exchange, observabilidade com Prometheus/Grafana e dashboards de negócio para o gestor.

Papeis dos Membros do Grupo

Membro	Papel principal	Contribuições
Membro 1	Co-líder análise	Levantamento bibliográfico e contexto de aplicação
Membro 2	Análise / Avaliação	Requisitos, referências teóricas, cenários de teste, análise crítica
Membro 3	Modelagem	Planta da oficina, C4 níveis 1 ^{textendash} 3, máquina de estados, sequence diagrams, deployment
Membro 4	Implementação	Código backend, workers, frontend, ajustes Docker Compose
Membro 5	Avaliação	Execução dos cenários, captura de logs, screenshots, análise dos resultados
Todos	Consolidação	Conclusão, revisão do relatório, slides, apresentação

Repositório: <https://github.com/<usuario>/projeto-final-sd>

References

- [1] A. S. Tanenbaum and M. Van Steen, *Distributed Systems: Principles and Paradigms*, 3rd ed. Pearson, 2017.
- [2] M. Fowler, “What do you mean by ‘Event-Driven’?,” <https://martinfowler.com/articles/201701-event-driven.html>, 2017.
- [3] Pivotal/VMware, “RabbitMQ documentation,” <https://www.rabbitmq.com/documentation.html>, 2024.
- [4] J. Kreps, N. Narkhede, and J. Rao, “Kafka: A distributed messaging system for log processing,” in *Proceedings of the NetDB Workshop*, 2011.
- [5] M. Richards and N. Ford, *Fundamentals of Software Architecture*. O’Reilly Media, 2020.
- [6] S. Brown, “The C4 Model for Software Architecture,” <https://c4model.com>, 2018.
- [7] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.
- [8] OASIS, “AMQP 0-9-1 Protocol Specification,” <https://www.amqp.org/specification/0-9-1/amqp-org-download>, 2008.
- [9] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge computing: Vision and challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637^{textendash}646, 2016.